

IF 260

Sistem Operasi

Minggu 3

Dr. Ananda Kusuma
e-mail: ananda.kusuma@lecturer.umn.ac.id

Universitas Multimedia Nusantara
Serpong, Tangerang

Agenda

- **Review:**
 - **System call**
- **Topik Minggu 3:**
 - **Process**
 - Model, penggunaan, states, implementasi
 - **Thread**
 - Model, penggunaan, states, implementasi

Review

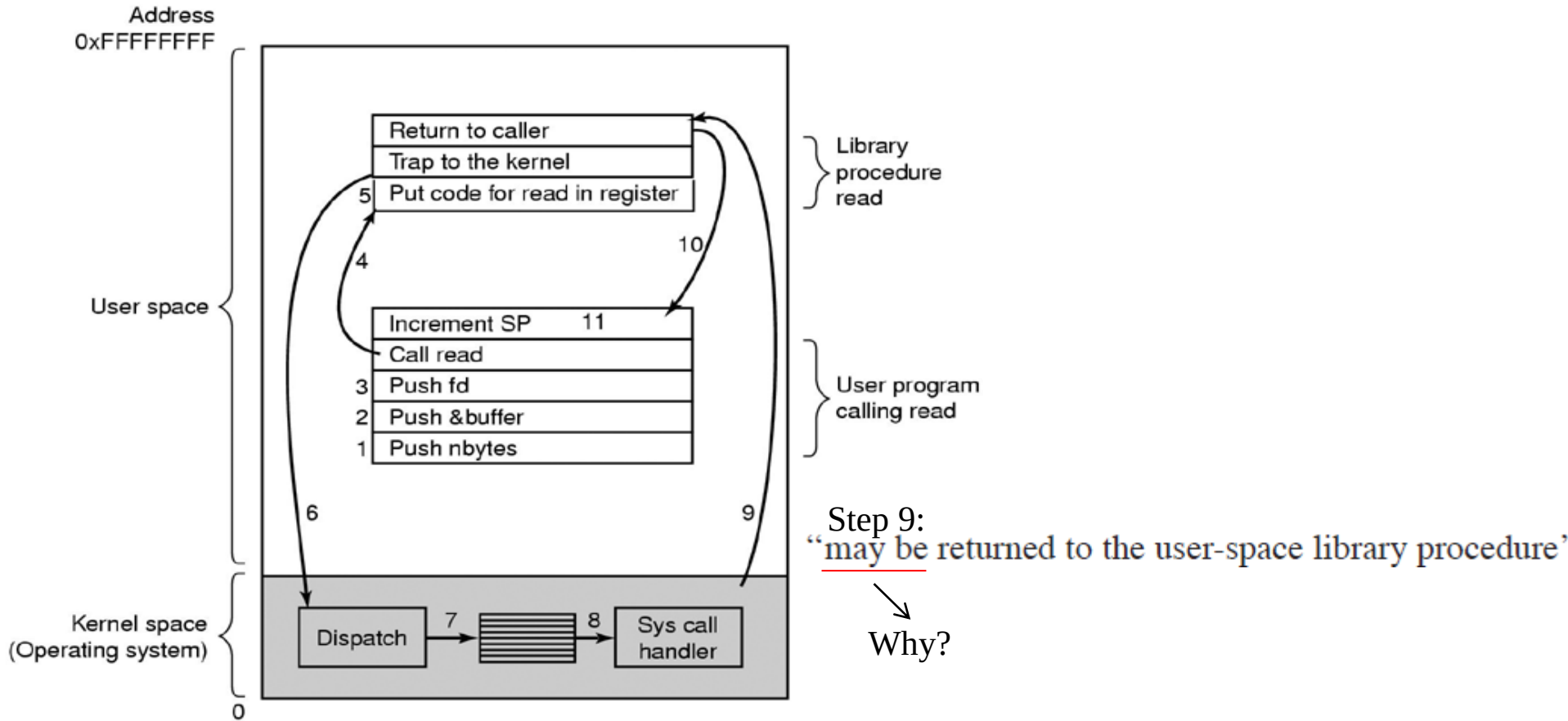
System Call (Trap Instruction)

Kenapa perlu system call?

- Apabila suatu user-level process memerlukan pengaksesan hardware via OS, maka process akan menjalankan suatu system call yang kemudian mengeksekusi suatu instruksi yang disebut trap instruction untuk memindahkan kendali ke kernel OS.
- Trap instruction adalah software interrupt (int 80h pada arsitektur x86) yang:
 - Switch status cpu ke supervisor mode
 - Mencari address dari fungsi kernel yang hendak dijalankan dari trap table
 - Menjalan fungsi kernel tersebut

Langkah-langkah dalam membuat system call

System call dari C program untuk membaca data dari file
`count = read(fd, buffer, nbytes)`



Process (Proses) (Lanjutan)

Process (Proses)

- **Process adalah**
 - Program yang sedang dieksekusi (berjalan)
 - Container yang berisikan seluruh informasi sumber daya yang diperlukan untuk menjalankan suatu program
 - Program : static file (image), misalnya executable file atau library file
 - Process : executing program = program + execution state
- **Tiap process memiliki PID (Process ID)**
 - Tiap process itu unik (PID berbeda menandakan process yang berbeda)
- **Program yang sama dapat dieksekusi beberapa kali, dan tiap eksekusi diabstrasikan dengan process yang berbeda.**
 - Abstraksi pemrosesan CPU pada beberapa program (multi-tasking) sehingga diperoleh nuansa seolah-olah program-program tersebut berjalan bersamaan (concurrency)

Process Resource Information (Informasi sumber daya proses)

- **Tiap process juga memiliki**
 - **Atribut/informasi pada process table atau process control block (PCB) yang berisikan berbagai sumber daya spt. Informasi registers (program counter, stack pointer), open files, alarm, process-process terkait, dsb.**

Process management	Memory management	File management
Registers Program counter Program status word Stack pointer Process state Priority Scheduling parameters Process ID Parent process Process group Signals Time when process started CPU time used Children's CPU time Time of next alarm	Pointer to text segment info Pointer to data segment info Pointer to stack segment info	Root directory Working directory File descriptors User ID Group ID

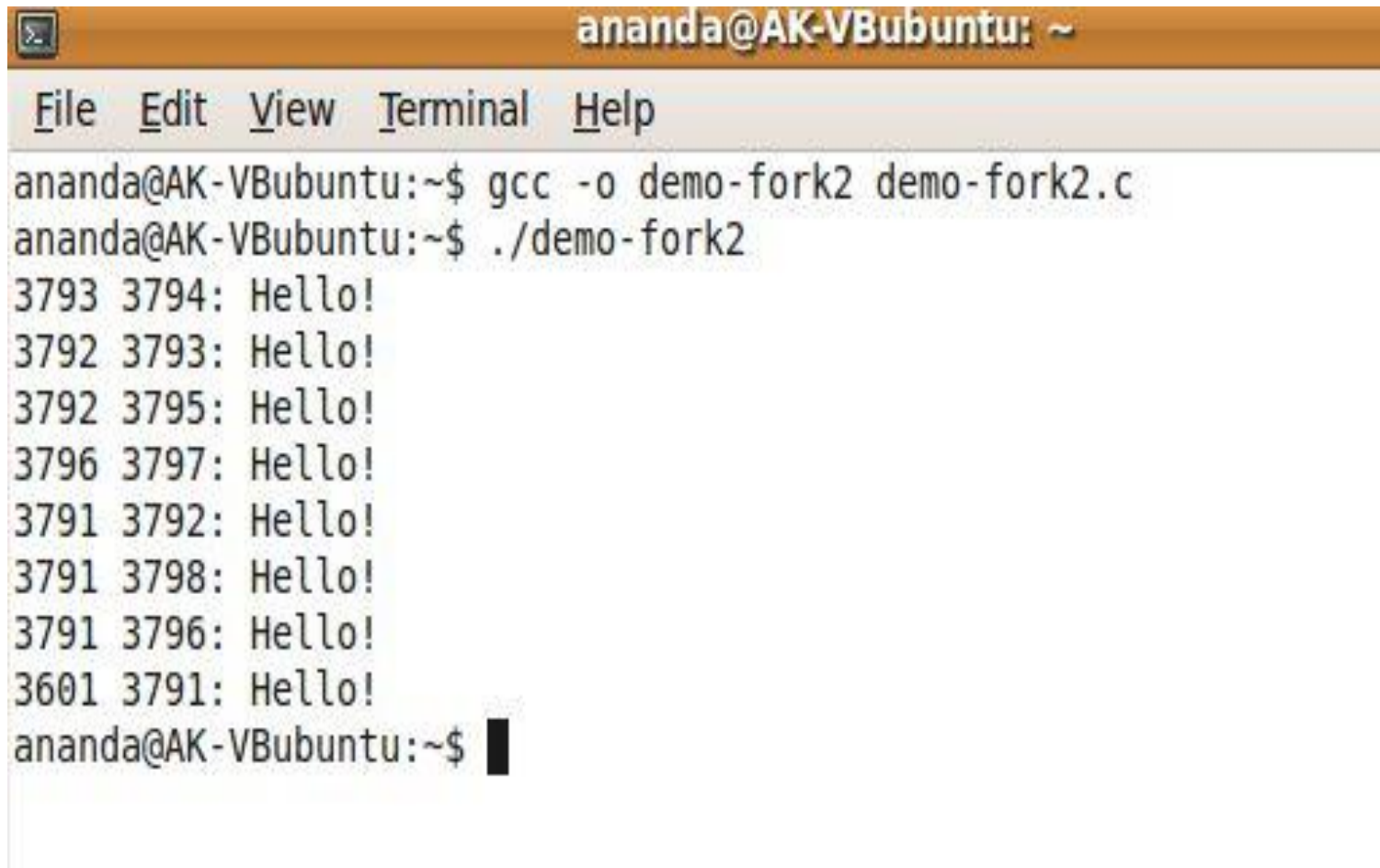
Process Creation (Pembuatan Proses)

- **Process dapat dibuat dan dihapus pada sistem secara dinamis**
- **Events yang menyebabkan pembuatan process**
 - **Inisialisasi sistem**
 - **Pengguna mengeksekusi command**
 - **Eksekusi system call, contoh: fork()**
 - **Inisiasi batch job**
- **Foreground vs background process**

Contoh: hirarki process dengan system call fork()

```
demo-fork2.c
1 #include <unistd.h>
2 #include <stdio.h>
3
4 int main()
5 {
6     fork();
7     fork();
8     fork();
9     printf("%d %d: Hello!\n",getppid(),getpid());
10 }
```

Contoh: hirarki process dengan system call fork()



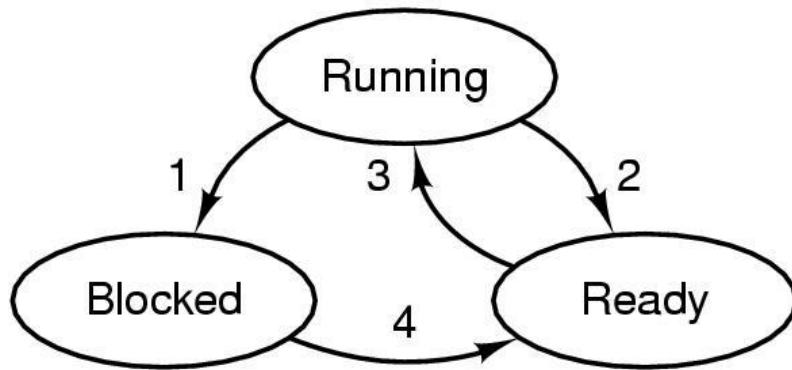
```
ananda@AK-VBubuntu: ~  
File Edit View Terminal Help  
ananda@AK-VBubuntu:~$ gcc -o demo-fork2 demo-fork2.c  
ananda@AK-VBubuntu:~$ ./demo-fork2  
3793 3794: Hello!  
3792 3793: Hello!  
3792 3795: Hello!  
3796 3797: Hello!  
3791 3792: Hello!  
3791 3798: Hello!  
3791 3796: Hello!  
3601 3791: Hello!  
ananda@AK-VBubuntu:~$
```

Process Termination (Penghentian Proses)

- **Kondisi yang dapat menyebabkan process berhenti**
 - **Normal exit (voluntary)**
 - **Error exit (voluntary)**
 - **Fatal error (involuntary)**
 - **Killed by another process (involuntary)**

Process States

- **Process memiliki execution states sbb.:**



1. Process blocks for input
2. Scheduler picks another process
3. Scheduler picks this process
4. Input becomes available

Ready: siap dieksekusi, menunggu penjadwalan CPU

Running: sedang dieksekusi oleh CPU

Blocked: sedang di-blocked, e.g. karena menunggu event IO atau karena sengaja di-blocked dgn syscall pause() atau, sleep(), dsb

Pertanyaan: bagaimana dengan process yang baru dibuat dan yang berhenti ?

Dispatcher: modul pada OS yang bertugas untuk switch processor dari process satu ke process yang lain

Contoh: process states (Linux)

Command:

```
$ ps aux
```

Beberapa states yang umum ditampilkan:

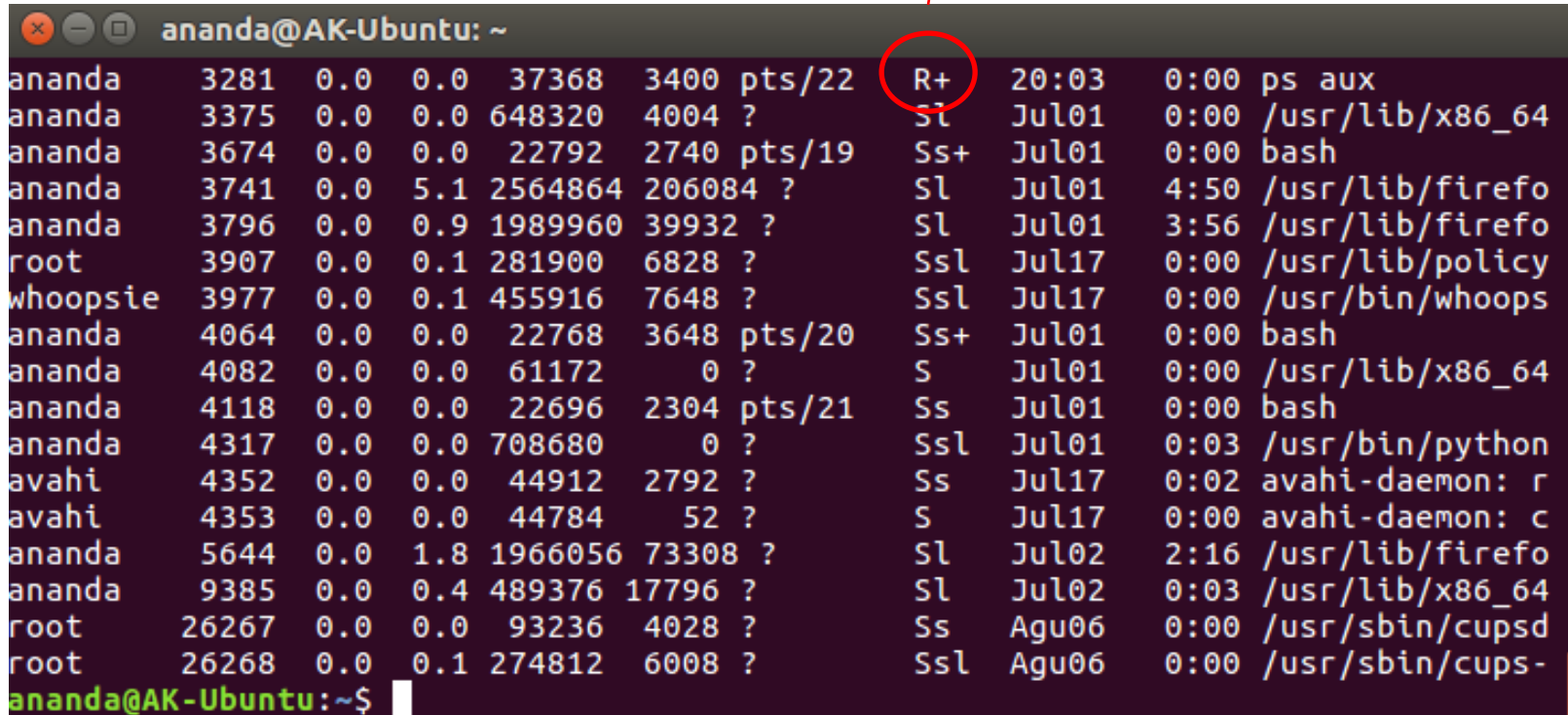
- R: running or runnable, it is just waiting for the CPU to process it
- S: Interruptible sleep, waiting for an event to complete, such as input from the terminal
- D: Uninterruptible sleep, processes that cannot be killed or interrupted with a signal, usually to make them go away you have to reboot or fix the issue
- Z: Zombie, we discussed in a previous lesson that zombies are terminated processes that are waiting to have their statuses collected
- T: Stopped, a process that has been suspended/stopped

Zombies and Orphans

- A child process whose parent has terminated is referred to as **orphan**
- When a child exits when its parent is not currently executing a `wait()`, a **zombie** emerges
- When a parent terminates, orphans are adopted by the `init` process of the system
- To see a zombie process: `ps -ef | grep defunct`

Contoh: process states (Linux)

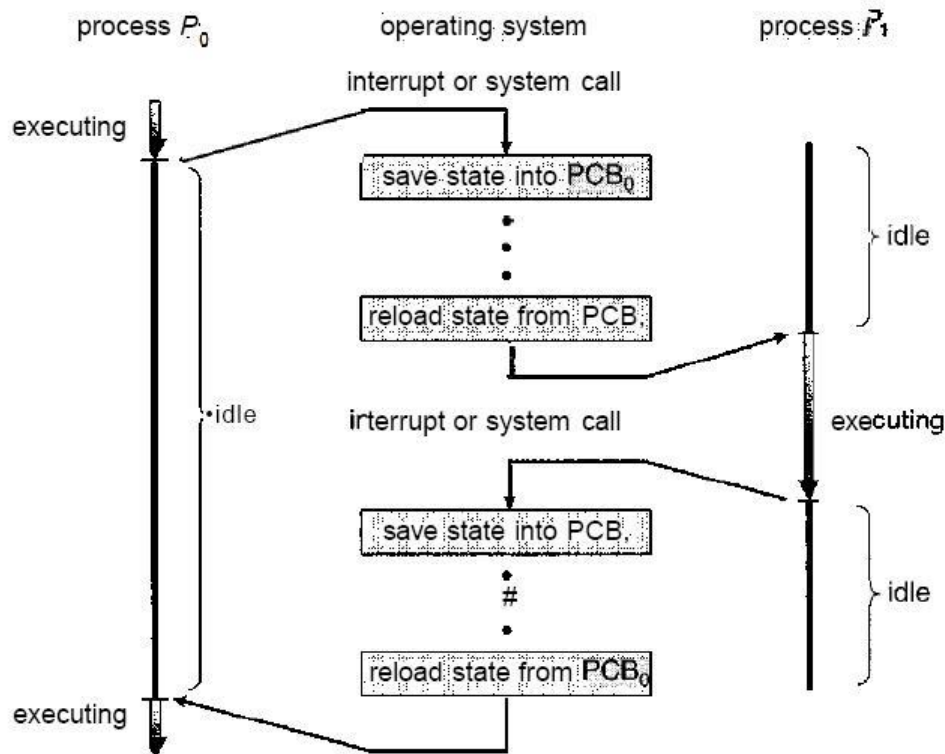
R+ : running or runnable in foreground



```
ananda@AK-Ubuntu: ~  
ananda 3281 0.0 0.0 37368 3400 pts/22 R+ 20:03 0:00 ps aux  
ananda 3375 0.0 0.0 648320 4004 ? Sl Jul01 0:00 /usr/lib/x86_64  
ananda 3674 0.0 0.0 22792 2740 pts/19 Ss+ Jul01 0:00 bash  
ananda 3741 0.0 5.1 2564864 206084 ? Sl Jul01 4:50 /usr/lib/firefo  
ananda 3796 0.0 0.9 1989960 39932 ? Sl Jul01 3:56 /usr/lib/firefo  
root 3907 0.0 0.1 281900 6828 ? Ssl Jul17 0:00 /usr/lib/policy  
whoopsie 3977 0.0 0.1 455916 7648 ? Ssl Jul17 0:00 /usr/bin/whoops  
ananda 4064 0.0 0.0 22768 3648 pts/20 Ss+ Jul01 0:00 bash  
ananda 4082 0.0 0.0 61172 0 ? S Jul01 0:00 /usr/lib/x86_64  
ananda 4118 0.0 0.0 22696 2304 pts/21 Ss Jul01 0:00 bash  
ananda 4317 0.0 0.0 708680 0 ? Ssl Jul01 0:03 /usr/bin/python  
avahi 4352 0.0 0.0 44912 2792 ? Ss Jul17 0:02 avahi-daemon: r  
avahi 4353 0.0 0.0 44784 52 ? S Jul17 0:00 avahi-daemon: c  
ananda 5644 0.0 1.8 1966056 73308 ? Sl Jul02 2:16 /usr/lib/firefo  
ananda 9385 0.0 0.4 489376 17796 ? Sl Jul02 0:03 /usr/lib/x86_64  
root 26267 0.0 0.0 93236 4028 ? Ss Agu06 0:00 /usr/sbin/cupsd  
root 26268 0.0 0.1 274812 6008 ? Ssl Agu06 0:00 /usr/sbin/cups-  
ananda@AK-Ubuntu:~$
```

Context Switching (Alih Konteks)

- Perpindahan eksekusi dari process satu ke process yang lain, yang meliputi langkah-langkah sbb:
 - Menghentikan jalannya suatu process dan menyimpan state/context CPU dari process tersebut ke suatu alamat di memory.
 - Menerima context process selanjutnya dari memory dan mengisi kembali CPU register
 - Memulai atau melanjutkan process dengan merujuk ke lokasi yang ditunjuk oleh program counter



- Mengambil sumber daya waktu
→ Extra latency
- Process context switching jauh lebih lambat dibandingkan thread switching

Contoh:

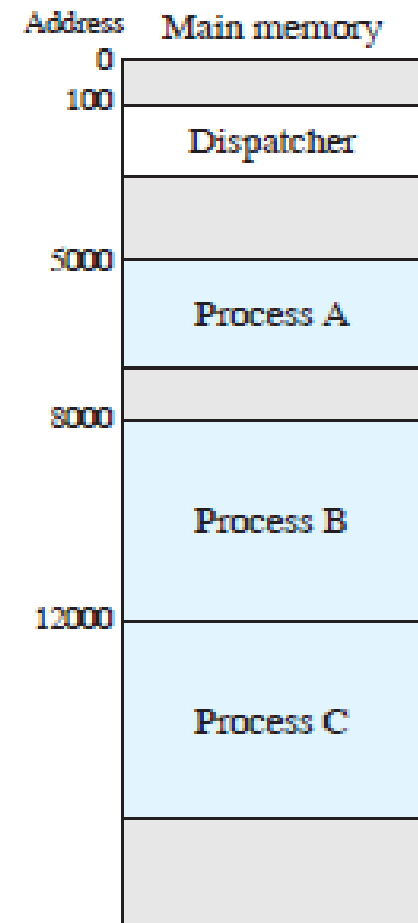
5000	8000	12000
5001	8001	12001
5002	8002	12002
5003	8003	12003
5004		12004
5005		12005
5006		12006
5007		12007
5008		12008
5009		12009
5010		12010
5011		12011

(a) Trace of Process A (b) Trace of Process B (c) Trace of Process C

5000 = Starting address of program of Process A

8000 = Starting address of program of Process B

12000 = Starting address of program of Process C



Contoh:

1	5000	27	12004
2	5001	28	12005
3	5002	-----Timeout	
4	5003	29	100
5	5004	30	101
6	5005	31	102
-----Timeout		32	103
7	100	33	104
8	101	34	105
9	102	35	5006
10	103	36	5007
11	104	37	5008
12	105	38	5009
13	8000	39	5010
14	8001	40	5011
15	8002	-----Timeout	
16	8003	41	100
-----I/O Request		42	101
17	100	43	102
18	101	44	103
19	102	45	104
20	103	46	105
21	104	47	12006
22	105	48	12007
23	12000	49	12008
24	12001	50	12009
25	12002	51	12010
26	12003	52	12011
		-----Timeout	

100 = Starting address of dispatcher program

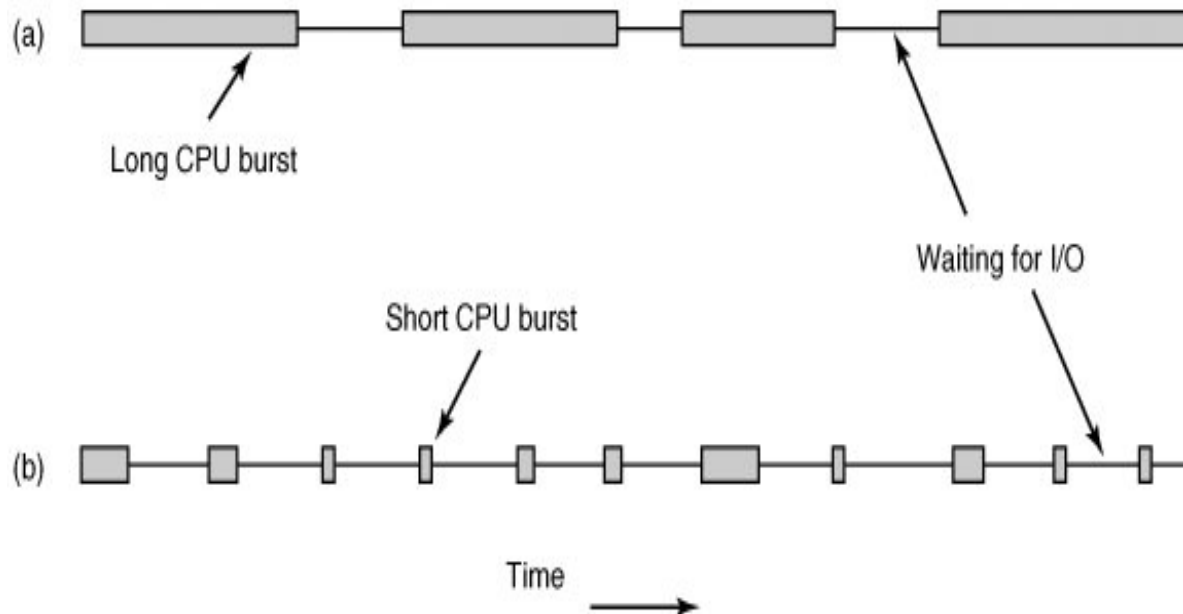
Shaded areas indicate execution of dispatcher process;

first and third columns count instruction cycles;

second and fourth columns show address of instruction being executed

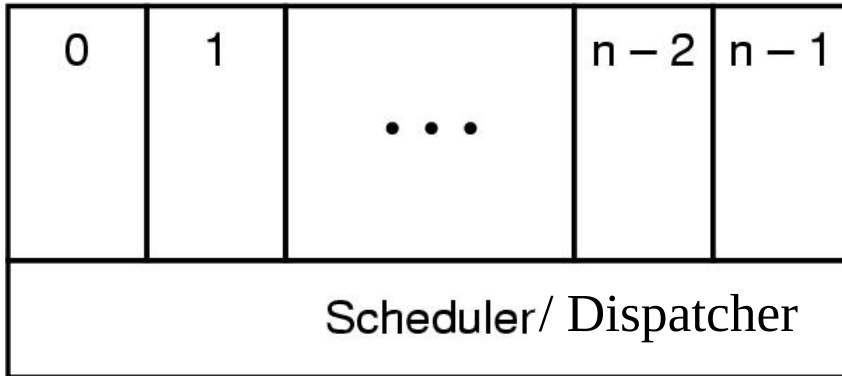
Process Behaviour (Perilaku Proses)

- **CPU-bound:**
 - Penyelesaian pekerjaannya ditentukan sepenuhnya oleh kecepatan CPU. Utilisasi CPU tinggi dan jeda waktu untuk pemrosesan layanan interrupt atau I/O peripherals rendah
- **IO-bound:**
 - Dibatasi oleh kecepatan pemrosesan layanan I/O. Umumnya ditandai oleh short CPU burst yang kemudian disertai waktu menunggu I/O. Utilisasi CPU rendah → multiprocessing dan threads

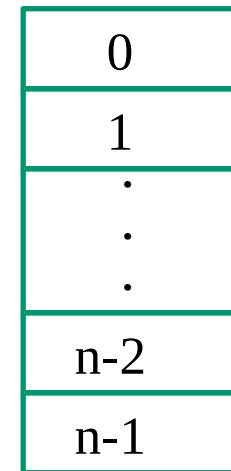


Konsep Multiprogramming

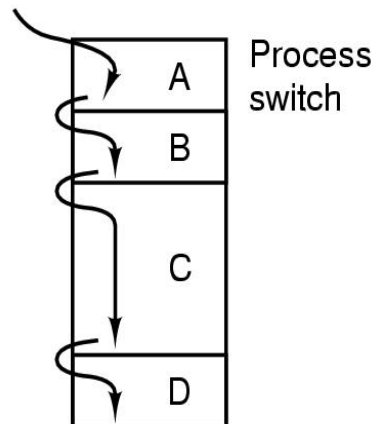
Processes



Address Space

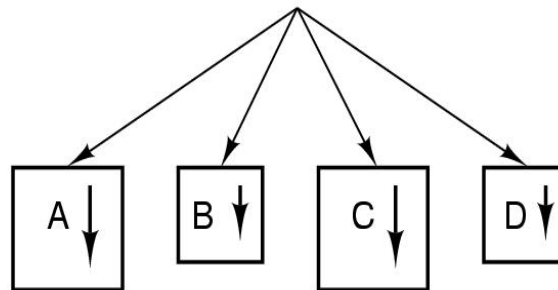


One program counter

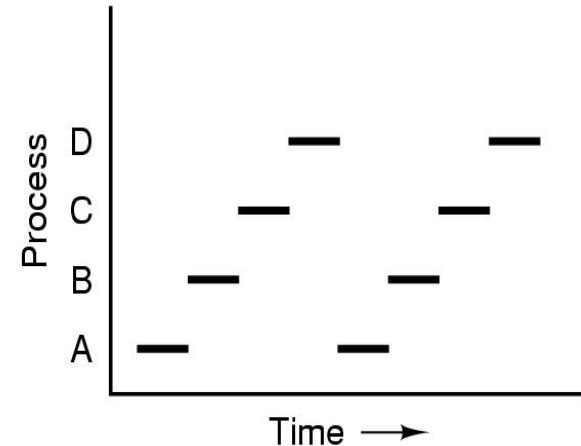


(a)

Four program counters



(b)



(c)

Review: Siswa dapat menjelaskan perbedaan monoprogramming, multiprogramming dan parallel programming

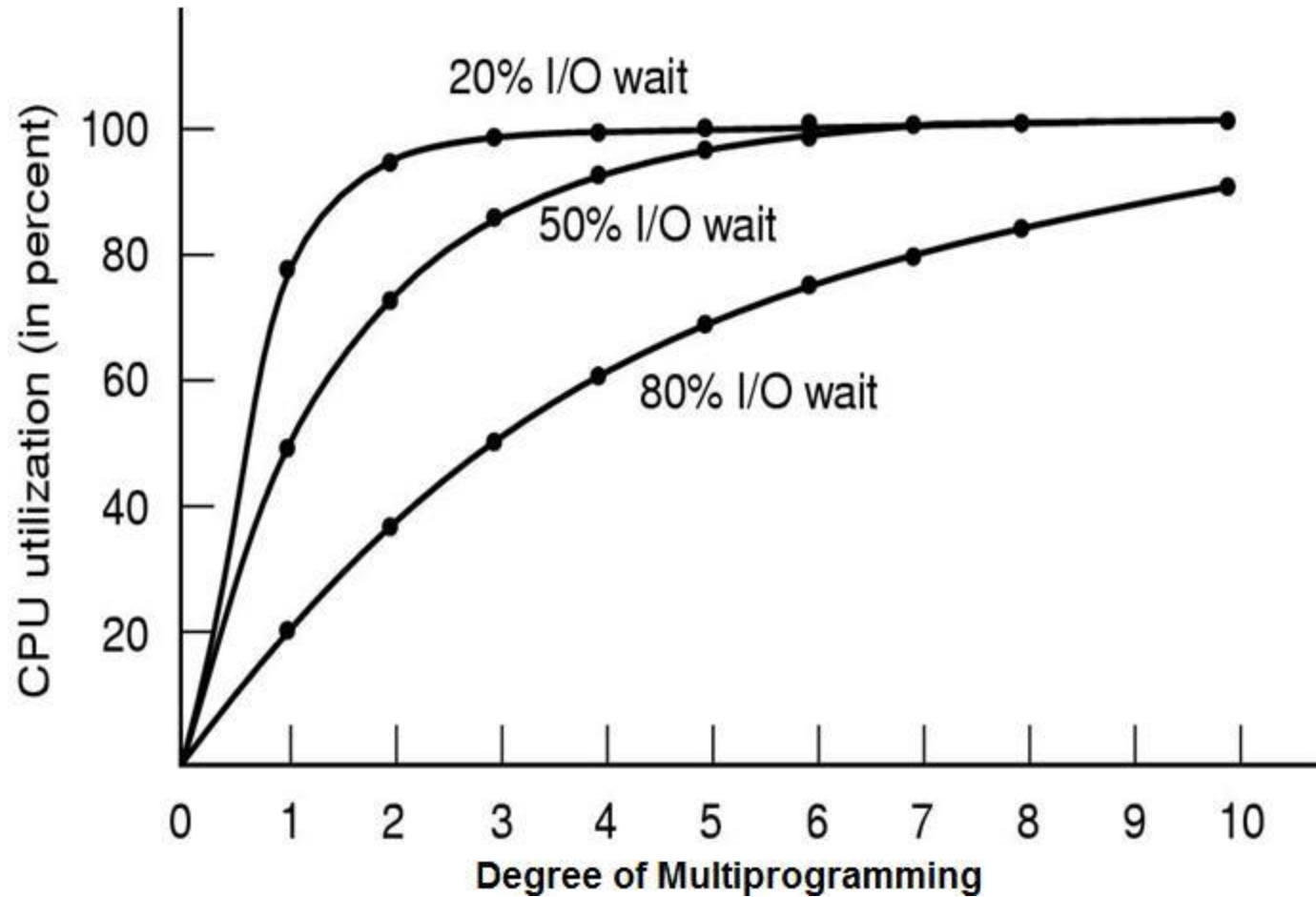
Modelling Multiprogramming

- Idealnya diperlukan analisa antrean (queuing analysis) untuk menghitung CPU utilization
 - interdependensi antar process → saat process ready, perlu antrean gilirannya untuk dijalankan oleh CPU
- Pendekatan dengan metode probabilistik sederhana yang mengasumsikan tiap process independent satu dengan yang lain dengan probability menunggu I/O (I/O wait) yang sama
 - Apabila p adalah probability menunggu waktu I/O tiap process, maka apabila ada n processes, probability menjadi p^n
 - Jadi:

$$\text{CPU utilization} = 1 - p^n$$

- Contoh soal: Jika I/O wait rata-rata dari suatu proses adalah 60%, dan jumlah proses yang running sebanyak 2 proses, maka utilisasi CPU adalah ?

CPU Utilization vs Degree of Multiprogramming

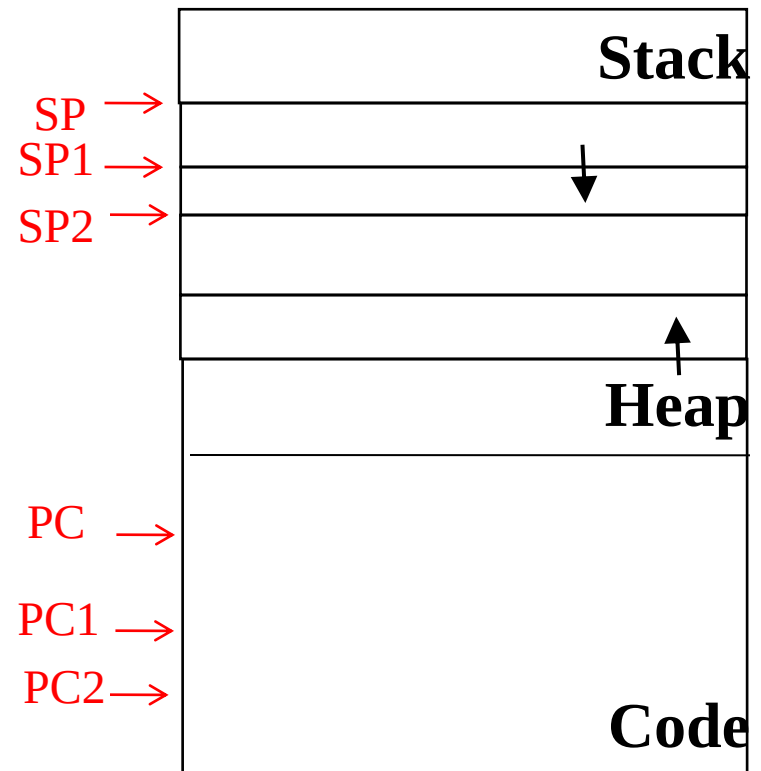


Thread

Threads (1)

- **Pengendali aliran program pada suatu process.**
 - Tiap process memiliki paling tidak satu thread
 - Process yang memiliki beberapa thread (multi-threaded process) memberikan tambahan nuansa concurrency pada tiap process
 - Disebut juga lightweight process, karena memiliki environment yang sama dengan process tapi dengan switching yang lebih sederhana

```
...  
int main() {  
    thrd1=pthread_create();  
    thrd2=pthread_create();  
    ...  
}
```

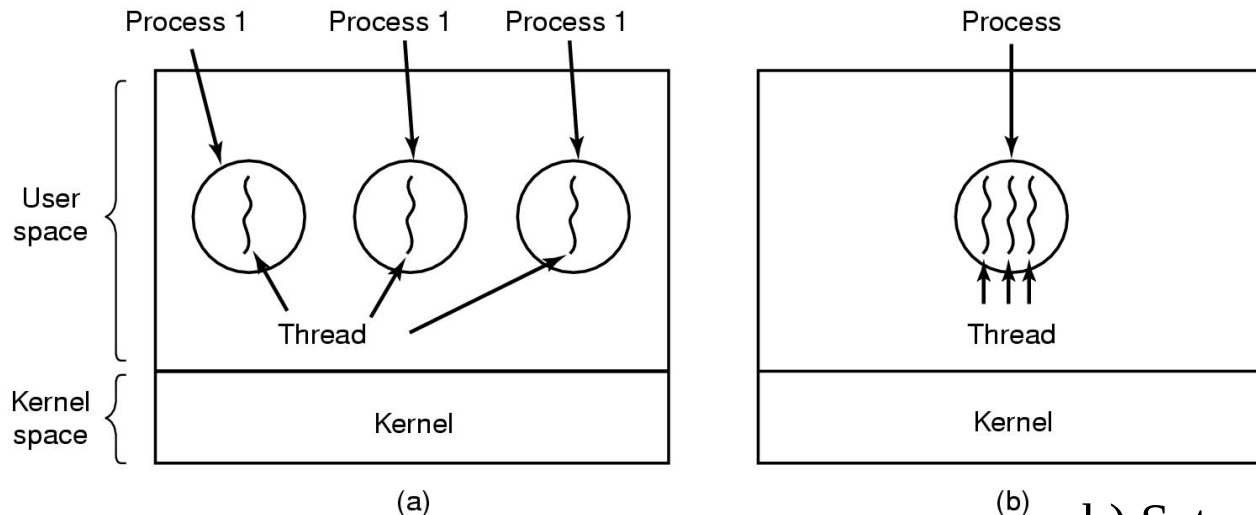


Threads (2)

- Resources (sumber daya) yang dimiliki process dan threads

Per process items	Per thread items
Address space Global variables Open files Child processes Pending alarms Signals and signal handlers Accounting information	Program counter Registers Stack State

- Classical Thread Model



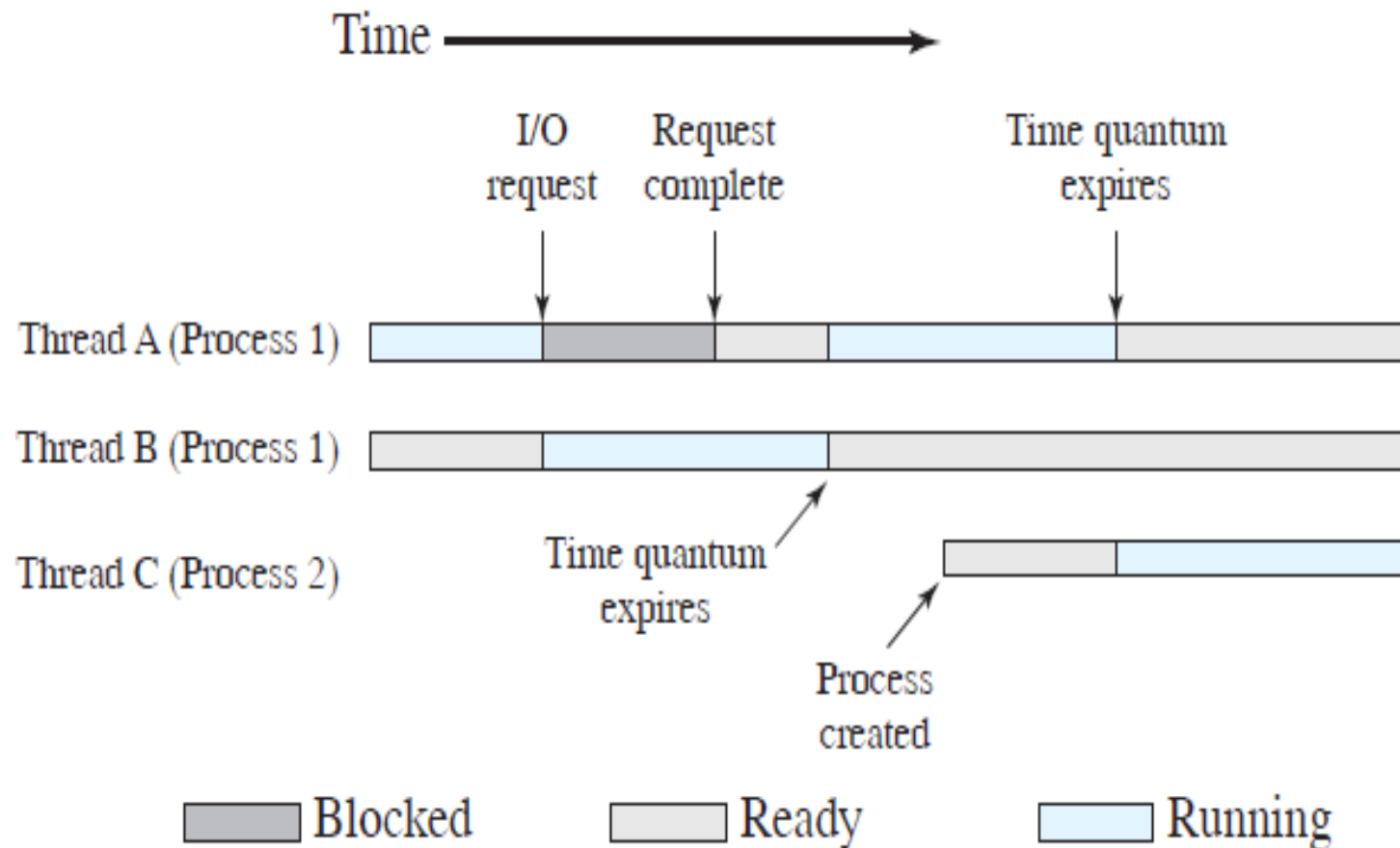
a) Tiga process masing-masing dengan satu thread

b) Satu process dengan tiga threads

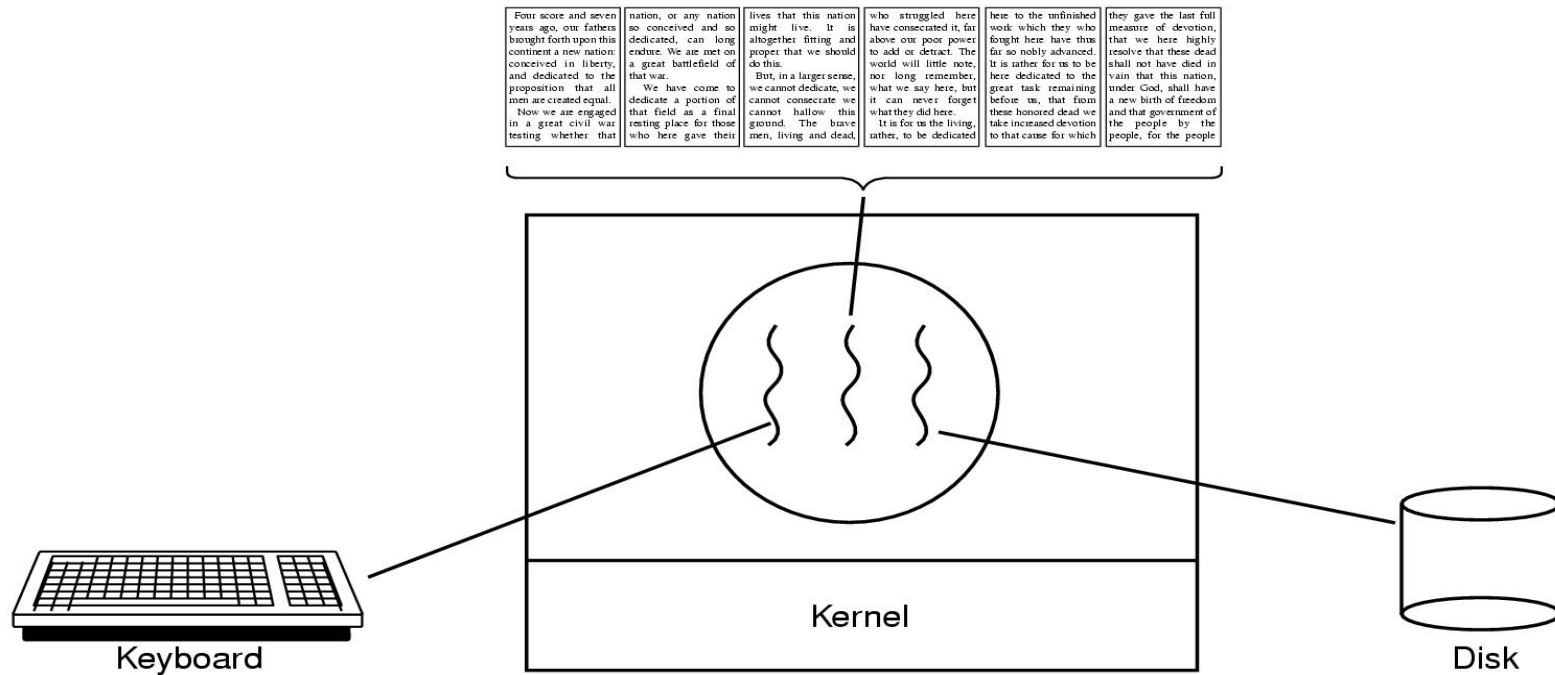
Alasan Pemakaian Threads

- Penambahan lingkungan concurrency eksekusi program pada tiap process, dengan tujuan meningkatkan CPU utilization
- Performance lebih cepat karena threads lebih mudah di-create dan di-delete dibandingkan processes.
- Komunikasi antar threads lebih mudah karena share address space yang sama
- Implementasi bisa di software (user level dan kernel level) dan juga hardware (sistem dengan multiple CPU)

Ilustrasi Multi-Threading pada Uniprocessor

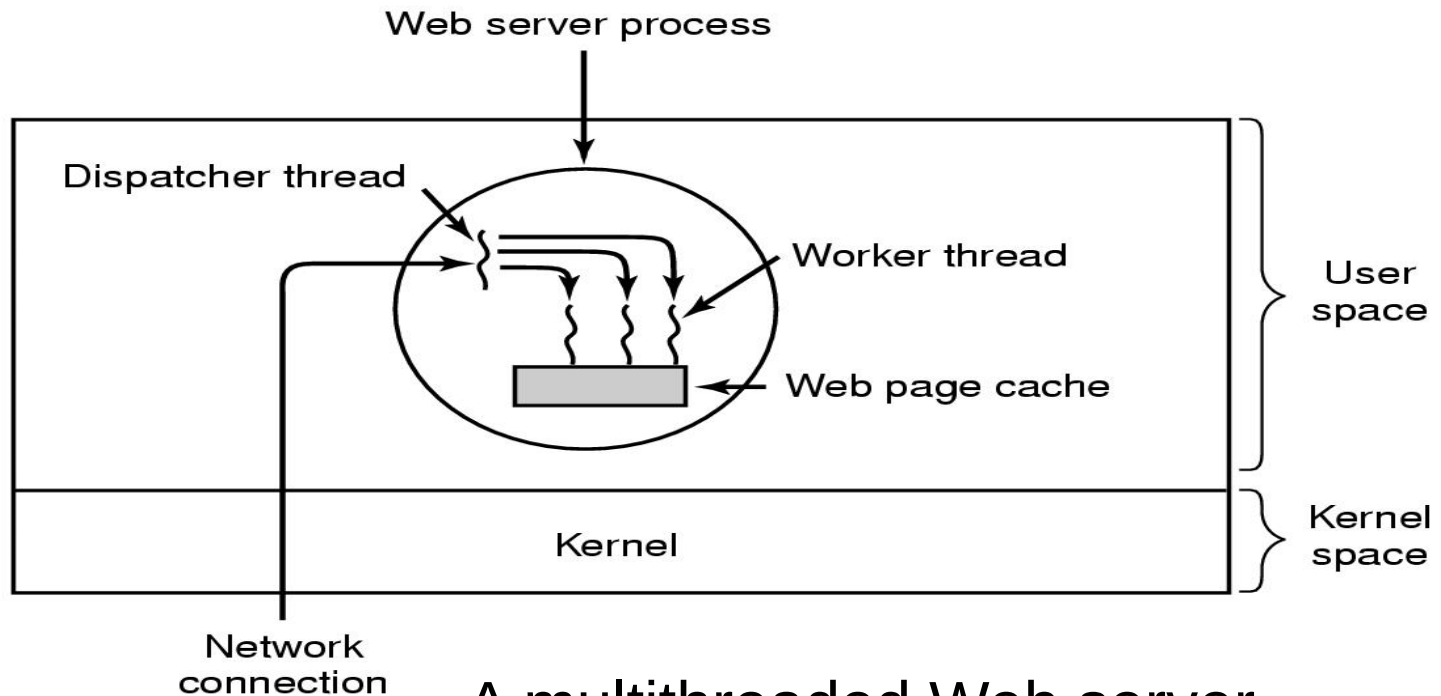


Contoh Penggunaan Threads (1)



A word processor with three threads.

Contoh Penggunaan Threads (2)



A multithreaded Web server.

Dispatcher
thread

```
while (TRUE) {  
    get_next_request(&buf);  
    handoff_work(&buf);  
}
```

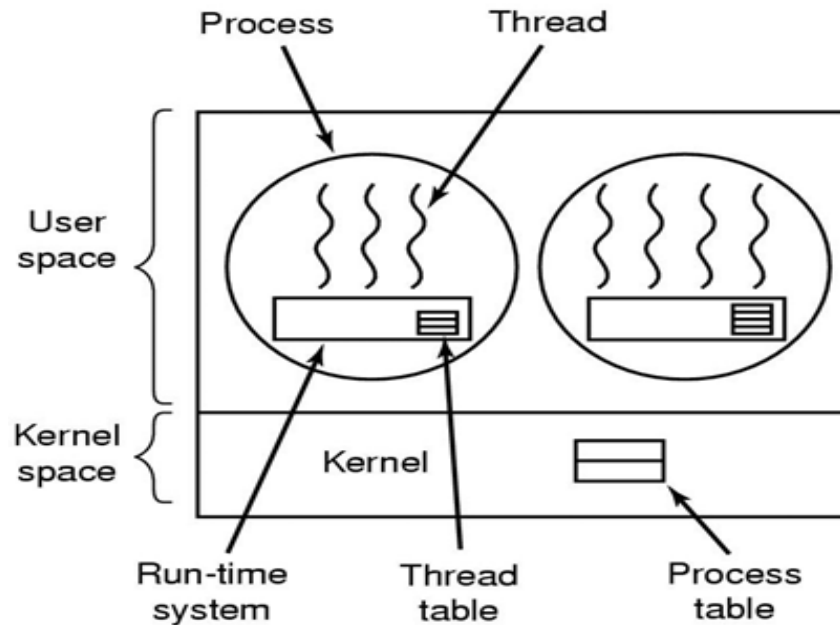
(a)

```
while (TRUE) {  
    wait_for_work(&buf)  
    look_for_page_in_cache(&buf, &page);  
    if (page_not_in_cache(&page)  
        read_page_from_disk(&buf, &page);  
    return_page(&page);  
}
```

(b)

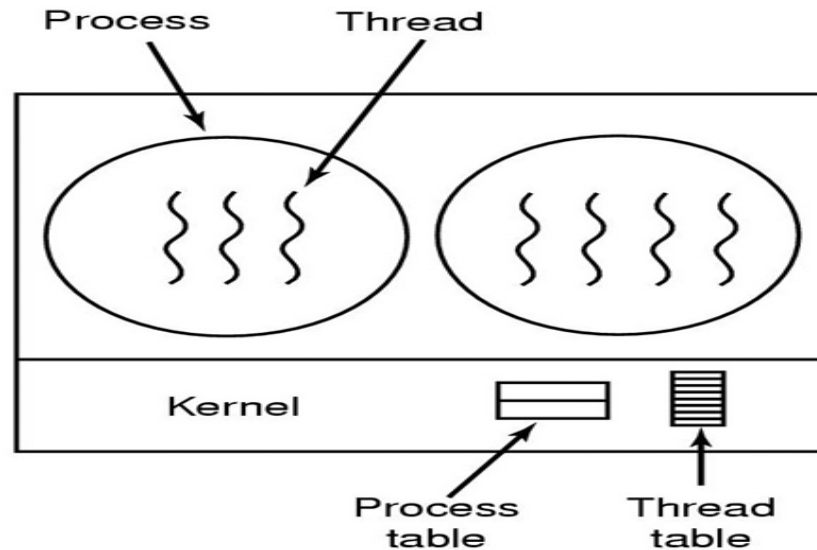
Worker
thread

Implementasi Threads pada User Space



- **Kelebihan:**
 - Performance, cepat untuk membuat dan switch thread
 - Proses dapat memiliki algoritma penjadwalan tersendiri (*customised*) untuk threads
 - Cocok untuk programming language-level threads, contoh Java
- **Kekurangan**
 - Masalah pada blocking system call, di mana threads justru sangat diperlukan

Implementasi Threads pada Kernel Space



- **Kelebihan:**
 - Tidak ada masalah dengan blocking system call karena kernel mengenal threads , dan bisa menjadwalkan ke thread lain di process yg sama atau yg lain
 - Mayoritas OS spt Windows, Linux, Solaris, dsb sudah menggunakan
 - Pthread lebih mengarah ke kernel-level thread
- **Kekurangan**
 - Overhead untuk membuat dan menghapus thread
 - solusi thread recycling
 - Hal-hal terkait penduplikasian process dan intersepsi signal antar process

Contoh Pemrograman Thread menggunakan pthread library

```
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>
#define NITERS 100000000
int cnt = 0;

void *count(void *arg)
{
    int i;

    for (i=0; i<NITERS; i++)
        cnt++;
}

int main()
{
    pthread_t tid1, tid2;

    pthread_create(&tid1, NULL, count, NULL);
    pthread_create(&tid2, NULL, count, NULL);
    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);
    if (cnt != (int)NITERS*2)
        printf("Salah... cnt = %d\n", cnt);
    else
        printf("Benar....cnt=%d\n", cnt);
}
```

Berapa jumlah thread?

Contoh:

- cpubound & iobound**
- thread**

Contoh cpubound process

```
cpubound.c
1 main()
2 {
3     int i=0;
4     for(;;){
5         i++;
6         i=i % 99999;
7     }
8 }
```

File Edit View Terminal Help											
top - 21:12:45 up 35 min, 4 users, load average: 1.08, 1.29, 1.16											
Tasks: 118 total, 3 running, 115 sleeping, 0 stopped, 0 zombie											
Cpu(s): 99.0%us, 1.0%sy, 0.0%ni, 0.0%id, 0.0%wa, 0.0%hi, 0.0%si, 0.0%st											
Mem: 1026680k total, 408060k used, 618620k free, 64140k buffers											
Swap: 915664k total, 0k used, 915664k free, 178068k cached											
PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
10017	ananda	20	0	1652	280	224	R	98.0	0.0	3:29.54	cpubound
2095	syslog	20	0	2040	716	564	S	0.3	0.1	0:00.15	syslogd
2585	root	20	0	5180	1908	1692	S	0.3	0.2	0:01.18	hald-addon-stor
2644	root	20	0	42936	15m	7436	S	0.3	1.5	0:24.62	Xorg
3566	ananda	20	0	34596	15m	9356	R	0.3	1.5	0:06.55	gnome-terminal
5194	ananda	20	0	2448	1192	912	R	0.3	0.1	0:03.09	top
1	root	20	0	3084	1888	564	S	0.0	0.2	0:01.99	init

Contoh iobound process

```
iobound.c
1
2 main()
3 {
4     int i=0;
5     for(;;)
6     {
7         i++;
8         i=i % 99999;
9         system("sleep 0.001s");
10    }
11 }
```

File Edit View Terminal Help

```
top - 21:05:31 up 27 min,  4 users,  load average: 2.52, 2.00, 1.20
Tasks: 125 total,  3 running, 121 sleeping,  0 stopped,  1 zombie
Cpu(s): 25.1%us, 69.1%sy,  0.0%ni,  4.8%id,  0.0%wa,  1.0%hi,  0.0%si,  0.0%st
Mem:   1026680k total,   356428k used,   670252k free,   15932k buffers
Swap:   915664k total,        0k used,   915664k free,   163612k cached
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
4052	ananda	20	0	1652	288	232	S	4.3	0.0	0:31.93	iobound
4	root	15	-5	0	0	0	S	1.7	0.0	0:08.54	ksoftirqd/0
3417	ananda	20	0	37044	19m	13m	S	1.0	1.9	0:11.76	gnome-panel
2644	root	20	0	42936	15m	7436	S	0.7	1.5	0:17.51	Xorg
3566	ananda	20	0	34468	14m	9356	R	0.7	1.5	0:04.29	gnome-terminal
14	root	15	-5	0	0	0	S	0.3	0.0	0:00.79	ata/0
3319	ananda	20	0	5736	1732	1332	S	0.3	0.2	0:02.91	VBoxClient
3380	ananda	20	0	29264	8888	6672	S	0.3	0.9	0:00.72	anome-setting

Tambahan 1 buah thread pada iobound process (total ada 2 buah thread)

```
iobound-1thread.c
1 #include <pthread.h>
2 #include <stdlib.h>
3 void task(int *j)
4 {
5     int i=*j;
6     for(;;)
7     {
8         i++;
9         i=i % 99999;
10    }
11 }
12 main()
13 {
14     int ret,i=0; pthread_t thrd1;
15     ret = pthread_create(&thrd1,NULL,(void *)task,(void *)&i);
16     if (ret) { perror("pthread_create: task\n"); exit(EXIT_FAILURE);}
17     for(;;)
18     {
19         i++;
20         i=i % 99999;
21         system("sleep 0.001s");
22     }
23 }
```

Tambahan 1 buah thread pada iobound process (total ada 2 buah thread)

File Edit View Terminal Help

```
top - 21:50:49 up 1:13, 5 users, load average: 2.90, 1.21, 0.57
Tasks: 120 total, 4 running, 116 sleeping, 0 stopped, 0 zombie
Cpu(s): 64.8%us, 34.9%sy, 0.0%ni, 0.0%id, 0.0%wa, 0.3%hi, 0.0%si, 0.0%st
Mem: 1026680k total, 417188k used, 609492k free, 65156k buffers
Swap: 915664k total, 0k used, 915664k free, 179092k cached
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
14082	ananda	20	0	10084	436	348	R	56.8	0.0	1:16.31	iobound-1thread
3319	ananda	20	0	5736	1732	1332	S	2.3	0.2	0:10.79	VBoxClient
2644	root	20	0	45632	17m	7696	S	2.0	1.8	0:50.18	Xorg
14	root	15	-5	0	0	0	S	0.3	0.0	0:01.78	ata/0
1992	root	20	0	3124	708	528	S	0.3	0.1	0:01.24	vboxadd-service
2452	haldaemo	20	0	6512	4416	3780	S	0.3	0.4	0:01.09	hald

Tambahan 2 buah thread pada iobound process (total ada 3 buah thread)

```
iobound-2thread.c
1 #include <pthread.h>
2 #include <stdlib.h>
3 void task(int *j)
4 {
5     int i=*j;
6     for(;;) {
7         i++;
8         i=i % 99999; }
9 }
10 main()
11 {
12     int ret,i=0;
13     pthread_t thrd1,thrd2;
14     ret = pthread_create(&thrd1,NULL,(void *)task,(void *)&i);
15     if (ret) { perror("pthread_create: task1\n"); exit(EXIT_FAILURE); }
16     ret = pthread_create(&thrd2,NULL,(void *)task,(void *)&i);
17     if (ret) { perror("pthread_create: task2\n"); exit(EXIT_FAILURE); }
18     for(;;)
19     {
20         i++;
21         i=i % 99999;
22         system("sleep 0.001s");
23     }
24 }
```

Tambahan 2 buah thread pada iobound process (total ada 3 buah thread)

File Edit View Terminal Help

```
top - 22:13:53 up 1:36, 5 users, load average: 1.44, 0.56, 0.81
Tasks: 121 total, 4 running, 117 sleeping, 0 stopped, 0 zombie
Cpu(s): 74.8%us, 25.2%sy, 0.0%ni, 0.0%id, 0.0%wa, 0.0%hi, 0.0%si, 0.0%st
Mem: 1026680k total, 419552k used, 607128k free, 65424k buffers
Swap: 915664k total, 0k used, 915664k free, 179044k cached
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
29947	ananda	20	0	18280	436	348	R	68.7	0.0	0:16.44	iobound-2thread
3598	ananda	20	0	16640	5208	3980	S	0.3	0.5	0:08.42	gnome-screensav
1	root	20	0	3084	1888	564	S	0.0	0.2	0:01.99	init
2	root	15	-5	0	0	0	S	0.0	0.0	0:00.04	kthreadd
3	root	RT	-5	0	0	0	S	0.0	0.0	0:00.00	migration/0
4	root	15	-5	0	0	0	S	0.0	0.0	0:09.97	ksoftirqd/0
5	root	RT	-5	0	0	0	S	0.0	0.0	0:00.00	watchdog/0

Akhir Kuliah Minggu 3

Terima kasih atas perhatiannya!